

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

THIẾT KẾ GIAO TIẾP BẤT ĐỒNG BỘ

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Asynchronous Message- Based Communication

Benefits and Challenges of Asynchronous Communication

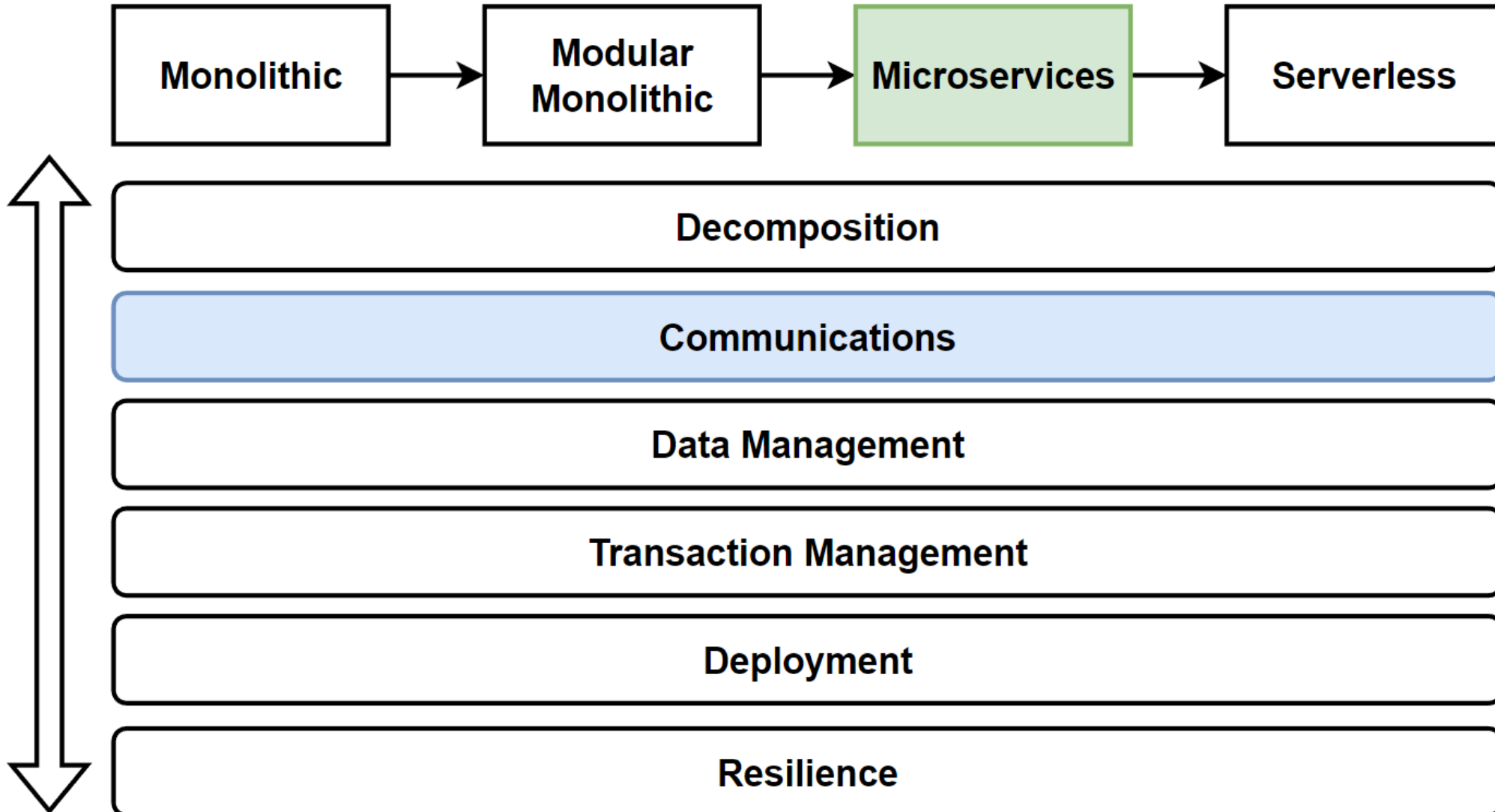
Asynchronous Message-Based Communication Types in Microservices Architecture

Single-receiver Message-based Communication (one-to-one model-queue)

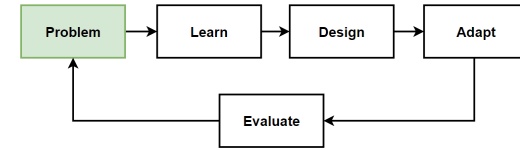
Multiple-receiver Message-based Communication (one-to-many model-topic)

Fan-Out Publish/Subscribe Messaging Pattern

Architecture Design – Vertical Considerations



Problem: Long Running Operations Can't Handle with Sync Communication



Problems

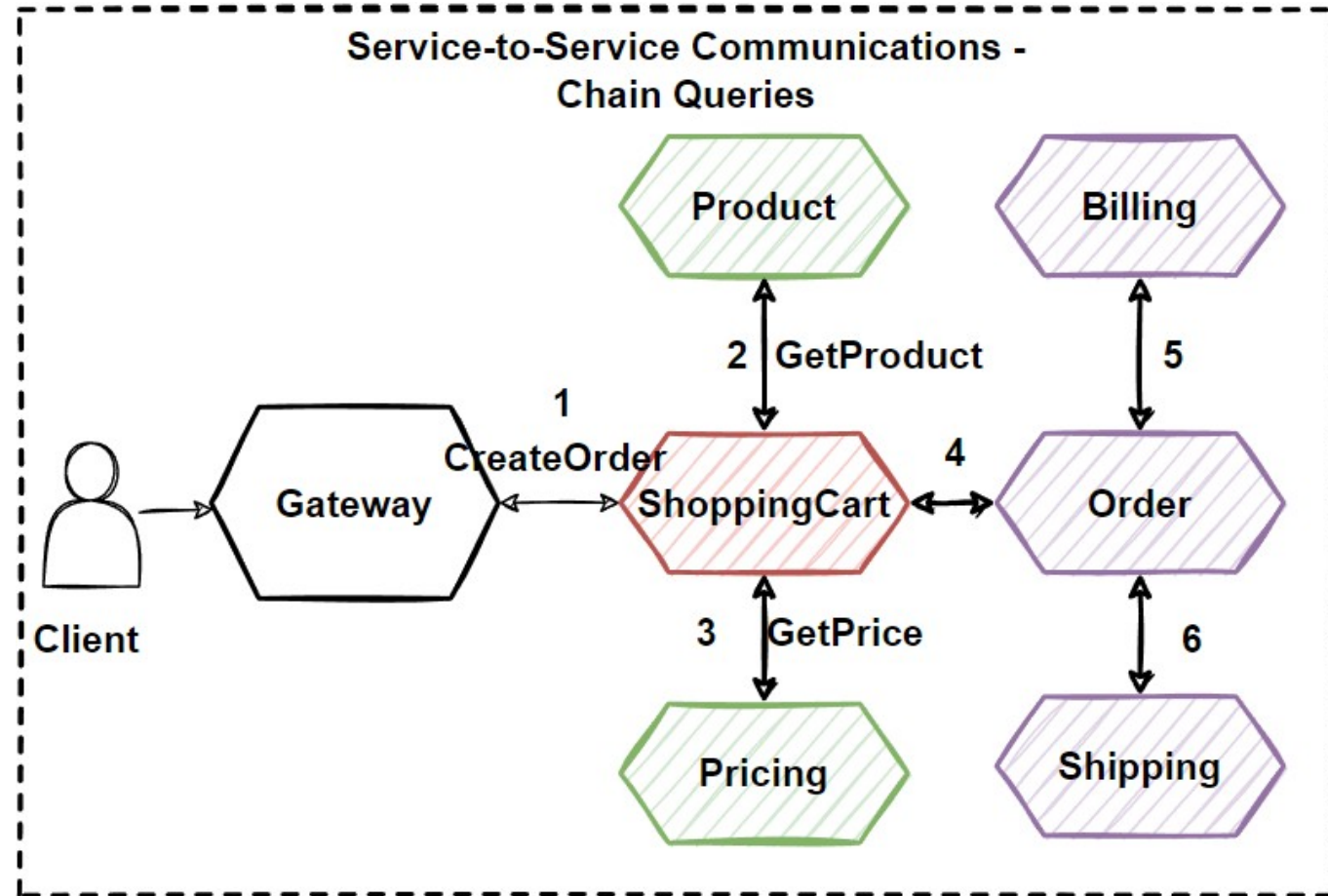
- HTTP calls to multiple microservices
- Chain Queries
- Visit more than a few microservices
- Increased latency with Highly Coupling Services
- Performance, scalability, and availability problems

Best Practices

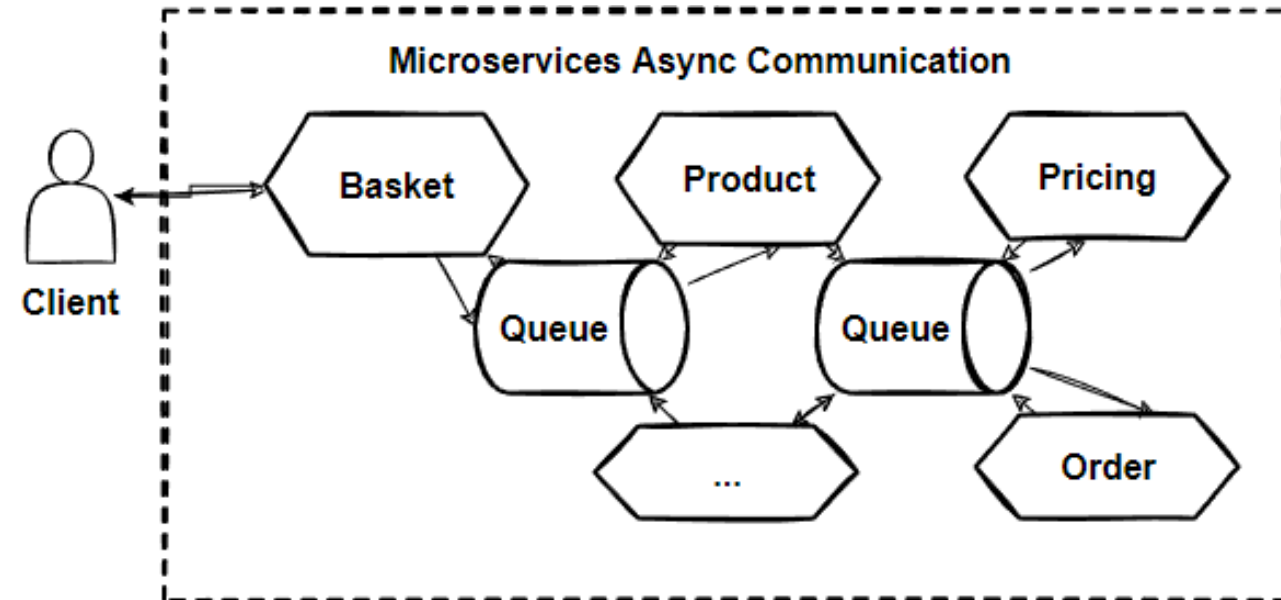
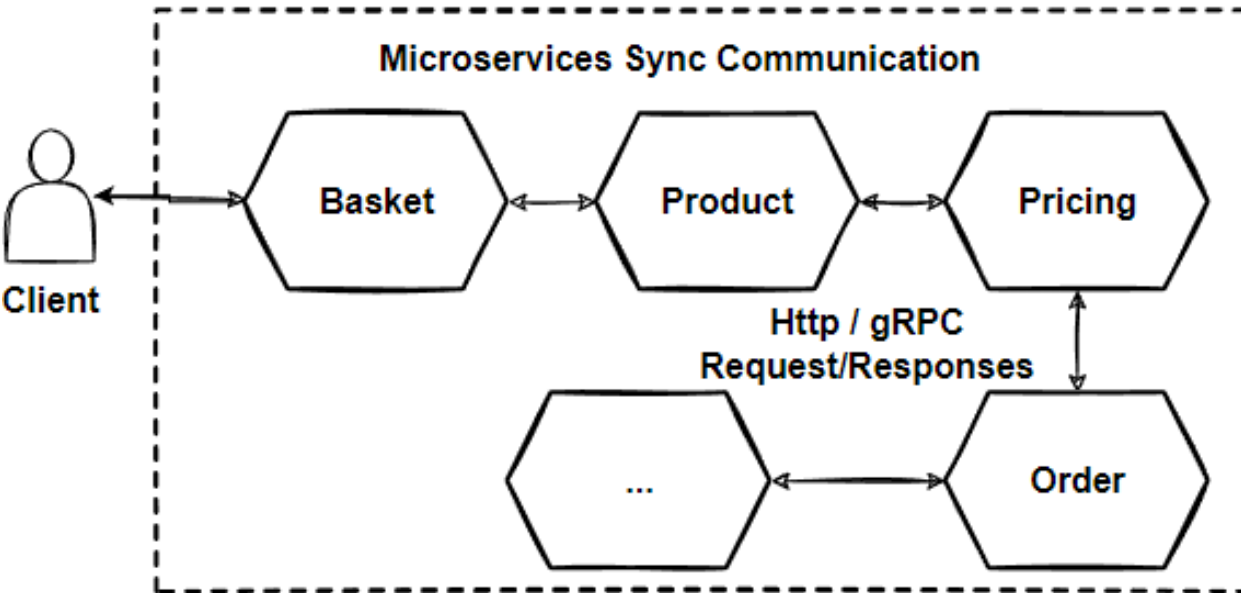
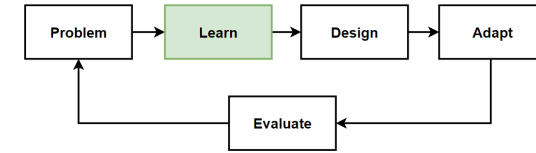
- Minimize the communication between the internal microservices
- Make microservices communication in Asynchronous way as soon as possible.

Solutions

- **Asynchronous Message-Based Communications**
- Working with events

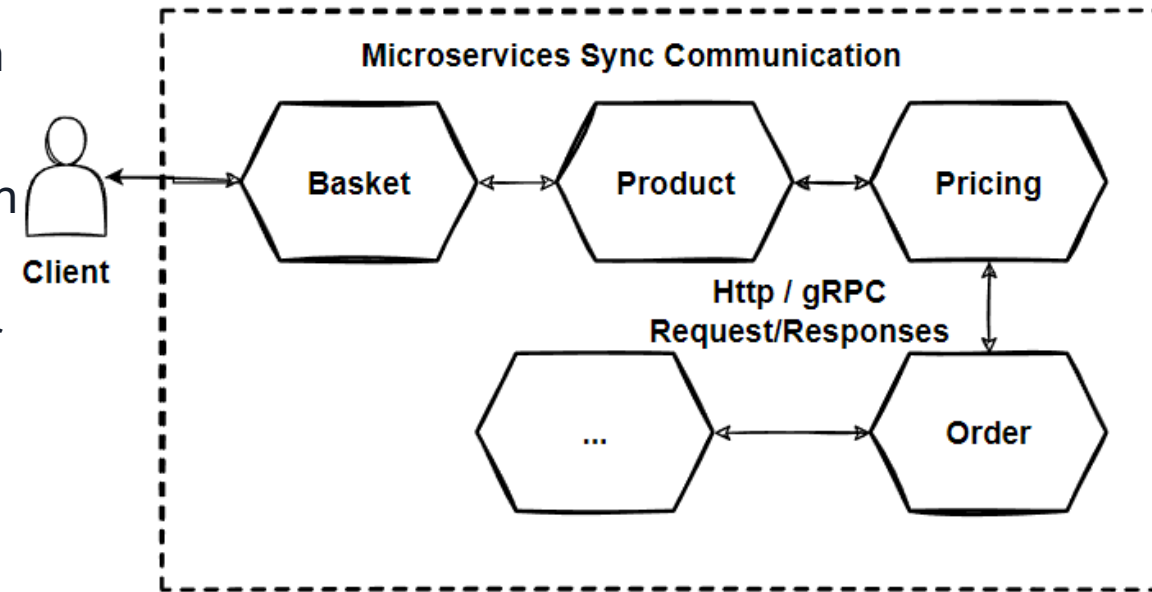
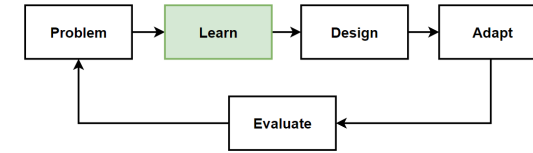


Microservices Communication Types - Sync or Async



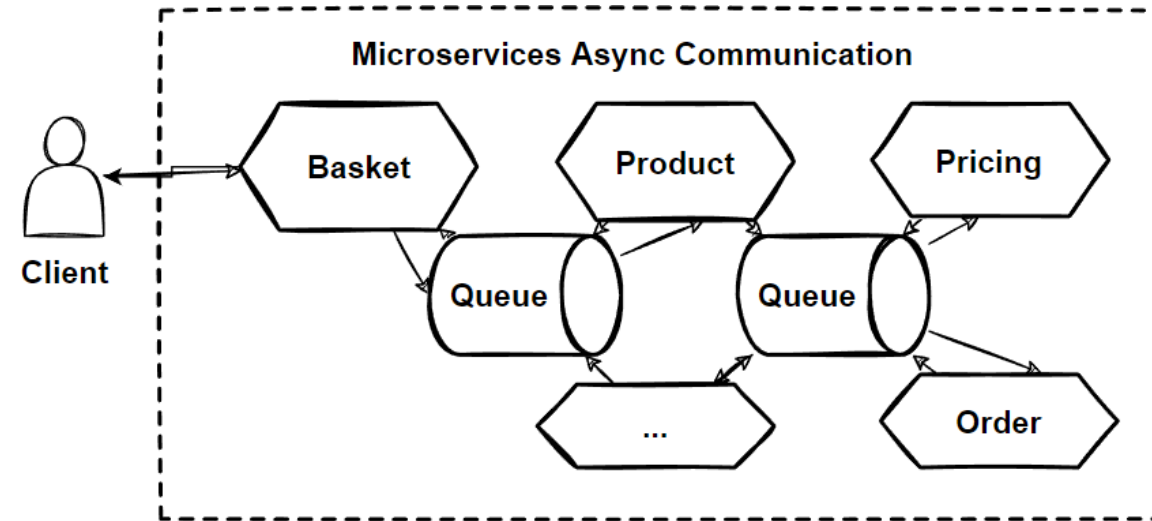
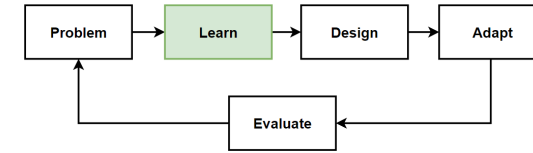
Microservices Synchronous Communication

- **Synchronous communication** is using **HTTP** or **gRPC** protocol for returning synchronous response.
- The client **sends a request** and **waits for a response** from the service.
- The client code **block their thread**, until the response reach from the server.
- The synchronous communication protocols can be **HTTP** or **HTTPS**.
- The client sends a request with using **http protocols** and waits for a response from the service.
- The client **call the server** and **block** client their operations.
- The client code will **continue** its task when it **receives** the HTTP server **response**.

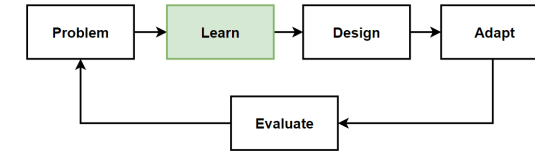


Microservices Asynchronous Communication

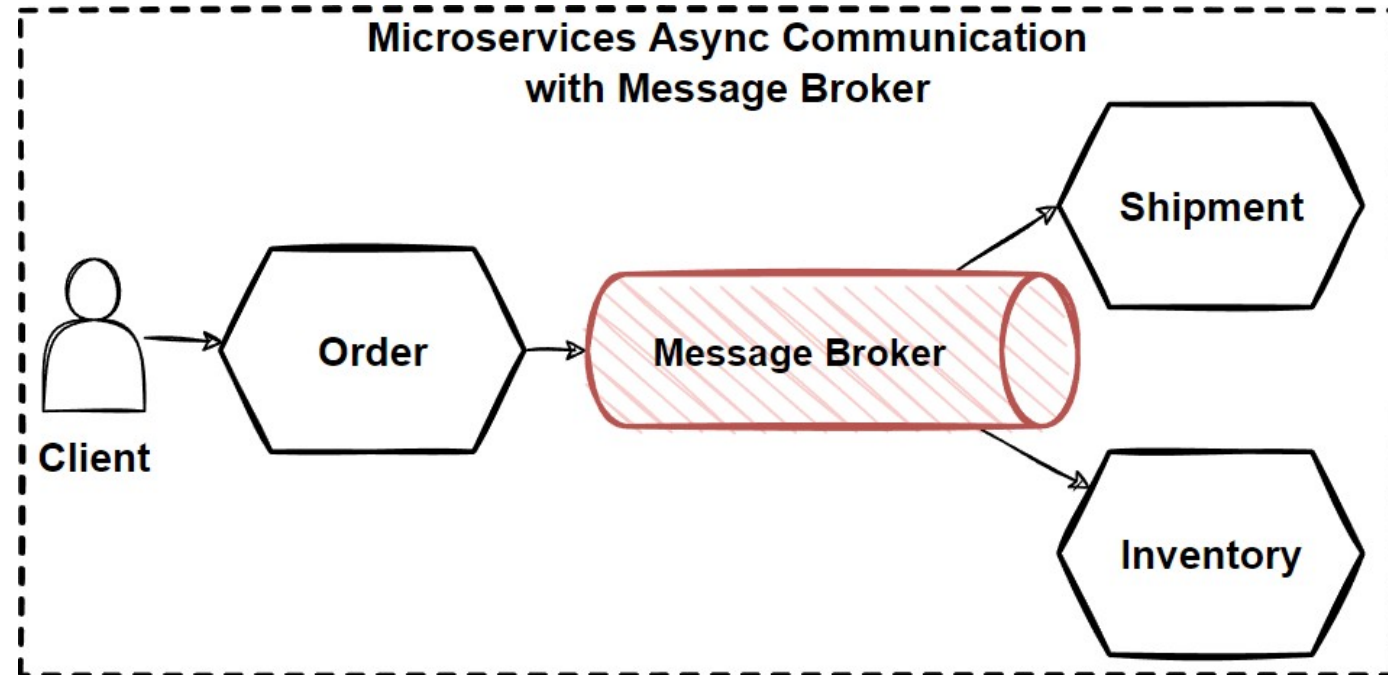
- The client sends a request but it **doesn't wait for a response** from the service. The client **should not have blocked** a thread while waiting for a response.
- **AMQP** (Advanced Message Queuing Protocol)
- Using AMQP protocols, the client **sends the message** with using message broker systems like **Kafka** and **RabbitMQ** queue.
- The message **producer does not wait** for a **response**.
- Message consume from the **subscriber** systems in async way, and no one waiting for response **suddenly**.
- If there is **busy interactions** in communication across multiple microservices, then use **asynchronous messaging platforms**.



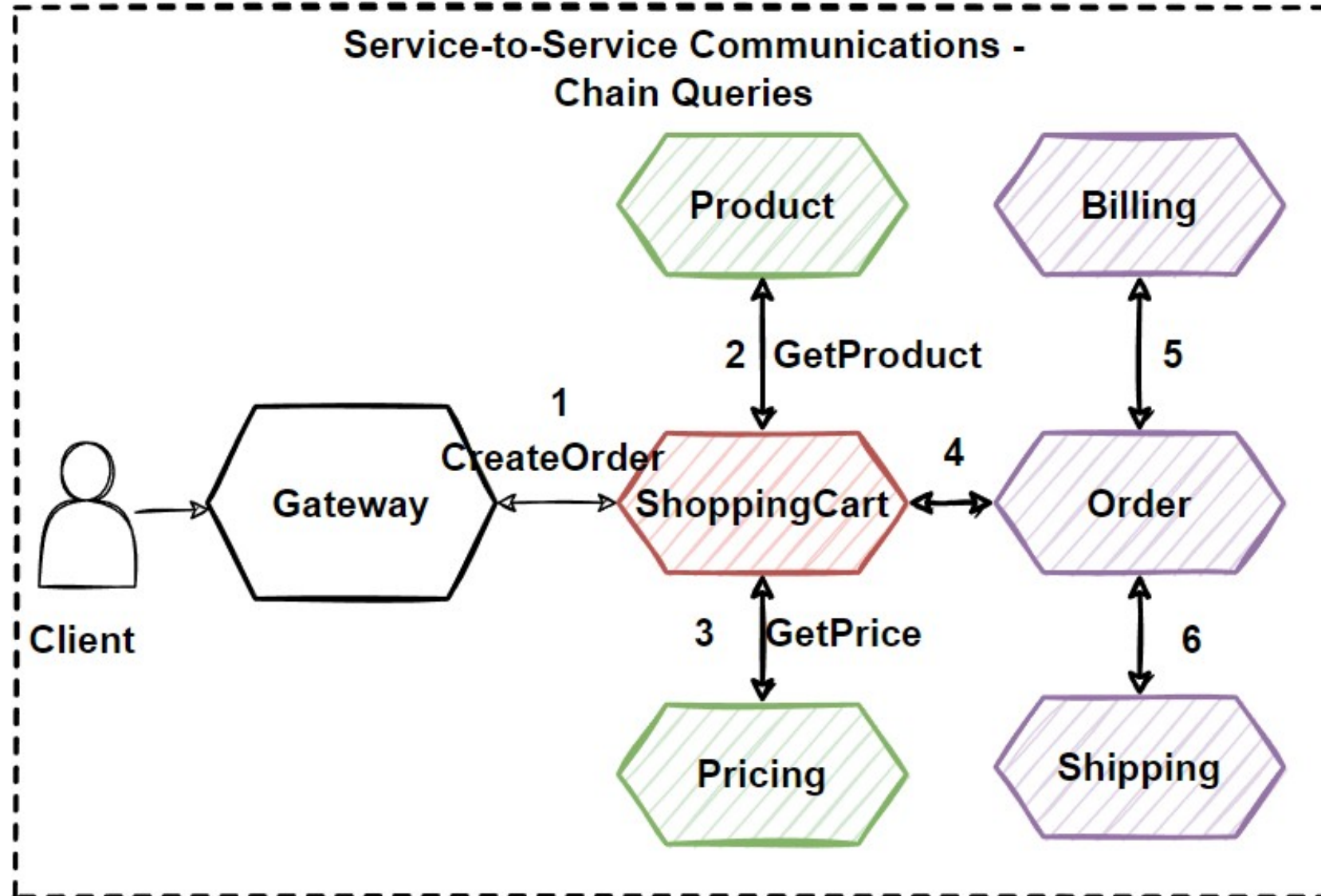
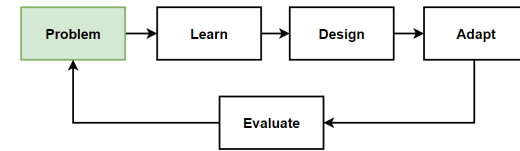
Microservices Asynchronous Communication-2



- **Message brokers** are responsible for handling the message sent by the producer service in async messaging-based communication.
- If the **consumer service** is **down** at the moment, the broker might be configured to **retry** as long as necessary for successful delivery.
- Messages can be **persisted if required** or stored only in memory.
- Message broker is **responsible** for **delivering** the message.
- No longer necessary for both microservices to be **up and running** for successful communication.
- Async messaging provides **loosely couple communication**.



Problem: Chain of request and highly coupled dependent microservices



Benefits of Asynchronous Communication

- **New Subscriber Services**

Adding new services is very simple. We can easily subscribe to message that we want to receive. The producer doesn't need to be know about subscribers, we can remove and add subscribers without affecting producer service.

- **Scalability**

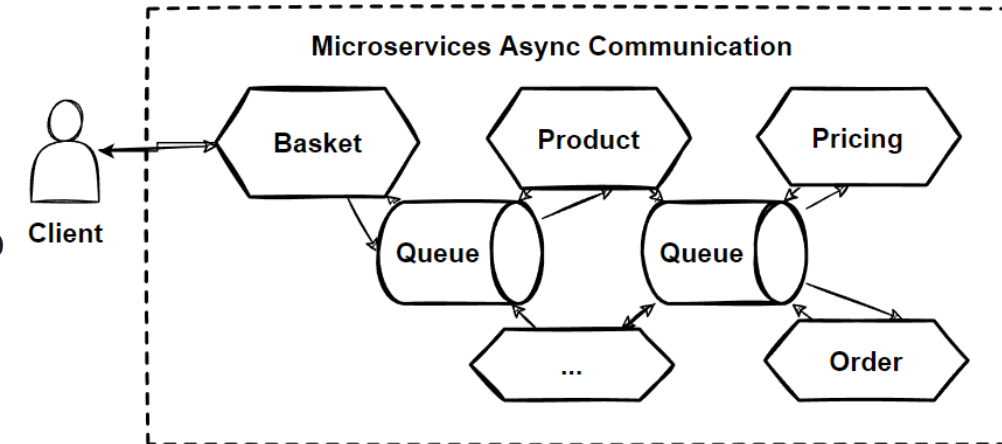
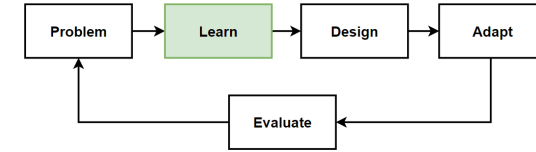
With async communications we can easier to manage scalability issues, can scale producer, consumer and message broker system independently. Scale services according to incoming messages into event bus system. Kubernetes KEDA Auto-scalers.

- **Event-driven Microservices**

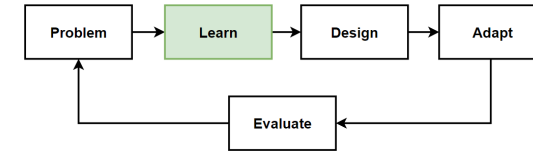
With async communication, we can provide event-driven architectures which is best way to communicate between microservices.

- **Retry mechanisms**

Brokers can retry to sending message and keep trying automatically without any custom solutions.



Challenges of Asynchronous Communication



- **Single Point of Failure - Message Broker**

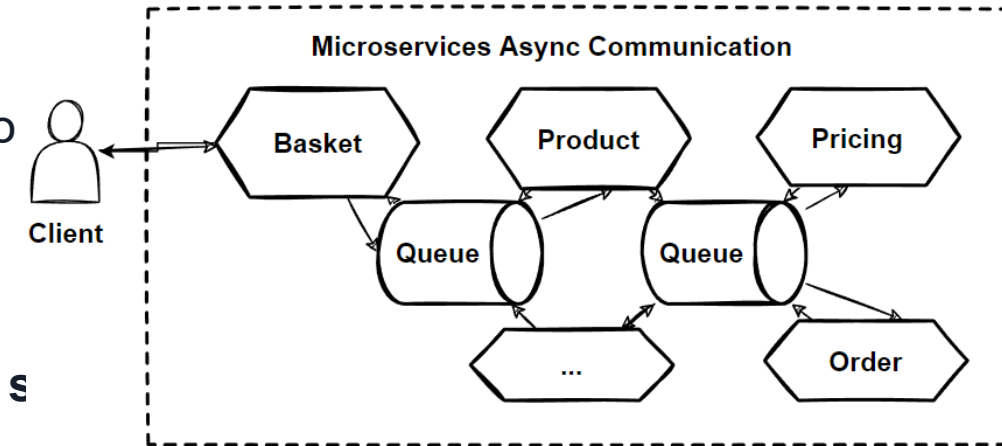
The message broker becomes a single point of failure. We should not rely of all communication with a single node of message brokers, instead we should scale it and use hybrid communication with sync and async in your cases.

- **Debugging**

difficult to debug issues with async communication, it can be hard to trace the flow of a single operation across service boundaries. debugging of the flow and the payload of events takes so many times and hard to debug at the same time.

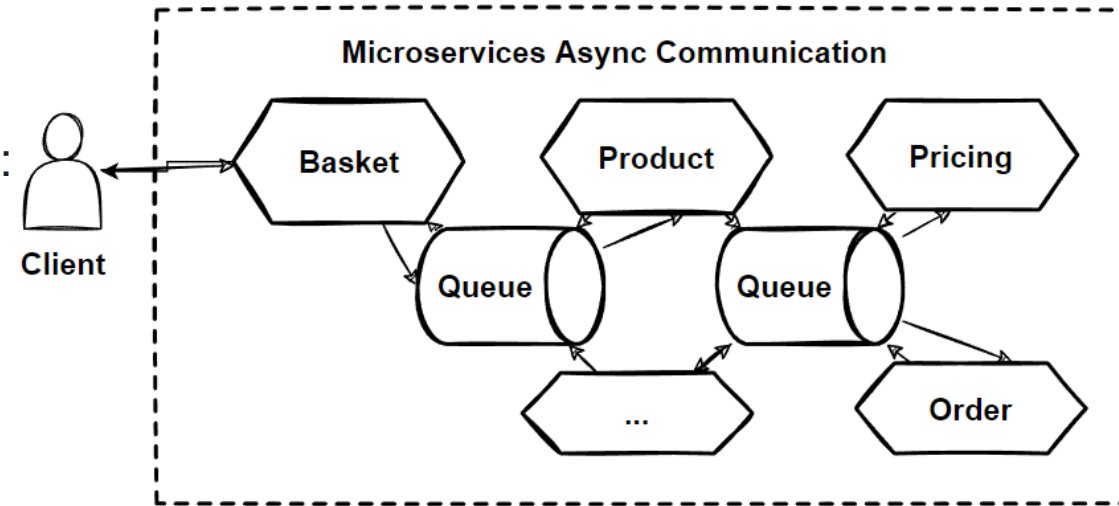
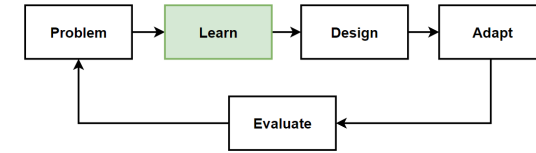
- **At-least-once delivery and Not Guarantee an order of message s**

Mostly Brokers use at-least-once delivery and not Guarantee order of messages. Should embrace these message delivery mechanism with applying idempotency consumers and not designing FIFO requires cases.

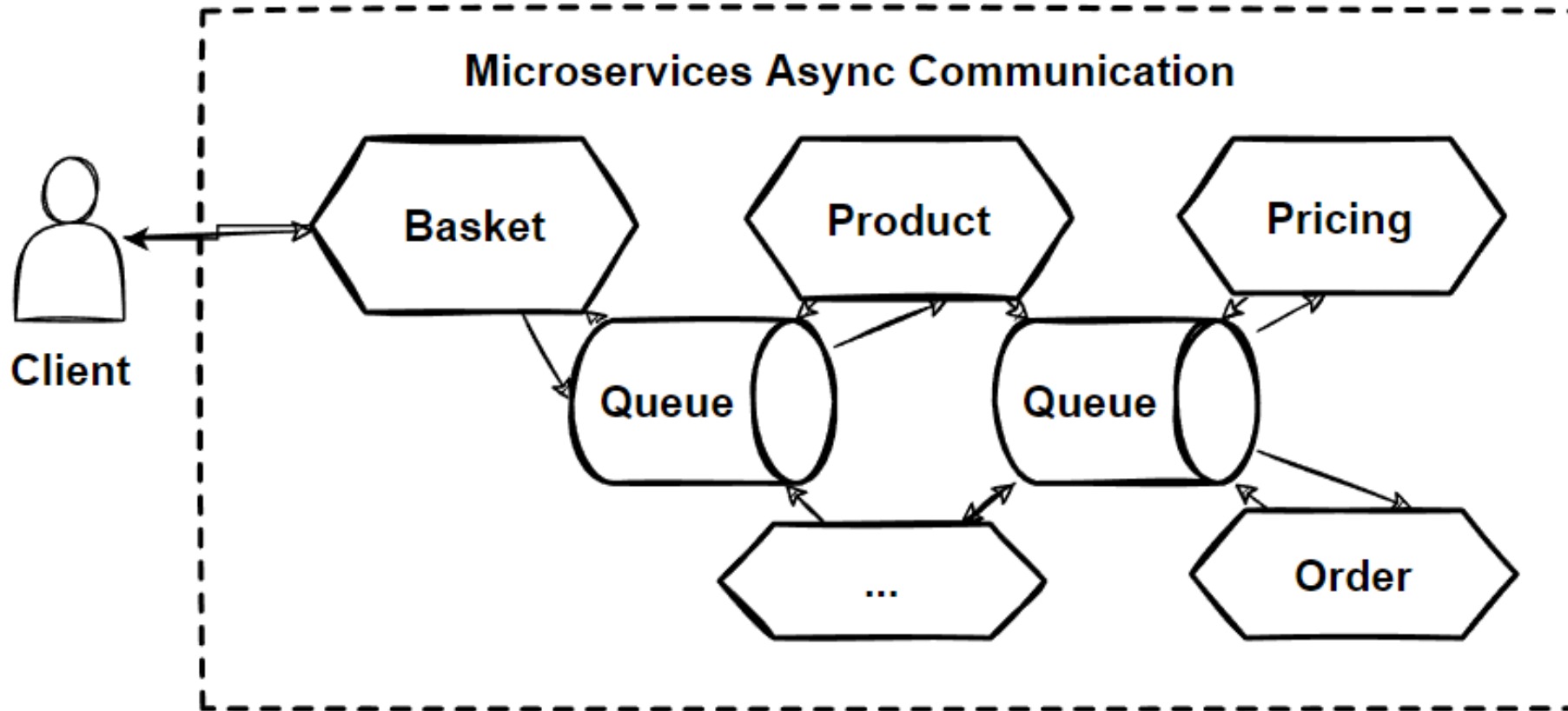
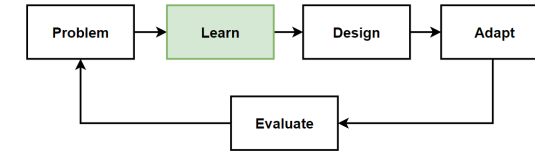


Asynchronous Message-Based Communication

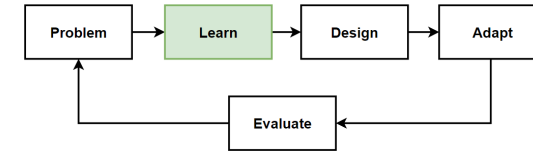
- Use **Asynchronous** message-based communication when you have **multiple microservices are required to interact each other** without any dependency.
- Asynchronous message-based communication works with **events**.
- **Events** can place the communication between microservices: **Event-driven communication**.
- If **any changes happens** in microservices, it is **propagating changes** across **multiple microservices** as an event.
- Events consumed by **subscriber microservices**.
- **Event-driven** communication brings us **Eventual consistency**.



Asynchronous Message-Based Communication-2



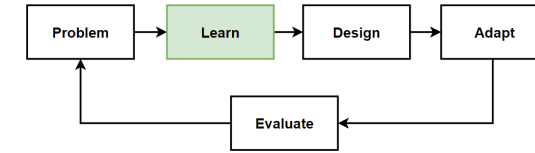
2 Type of Asynchronous Messaging Communication



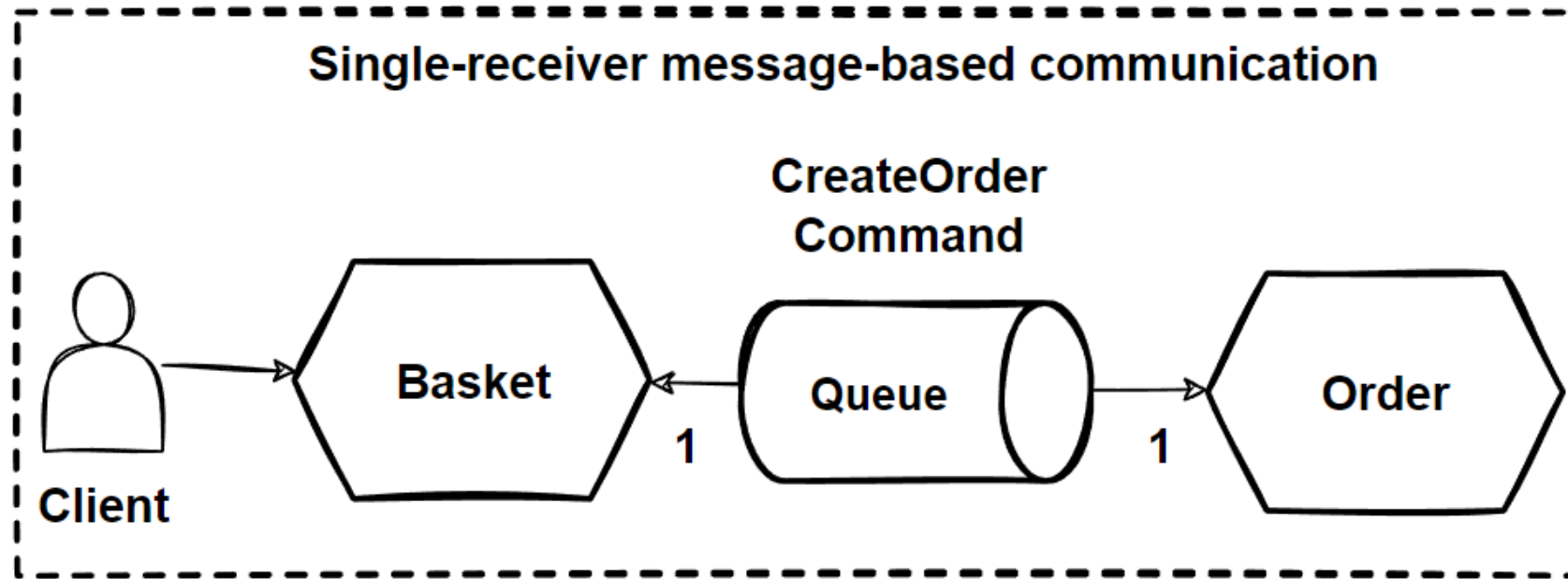
**Single receiver message-based communication
one-to-one(queue) model or
Point-to-point model**

**Multi receiver message-based communication
one-to-many (topic) model or
publish/subscribe model**

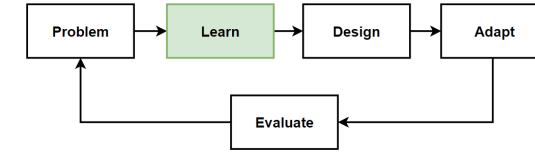
Single-receiver Message-based Communication 1-1



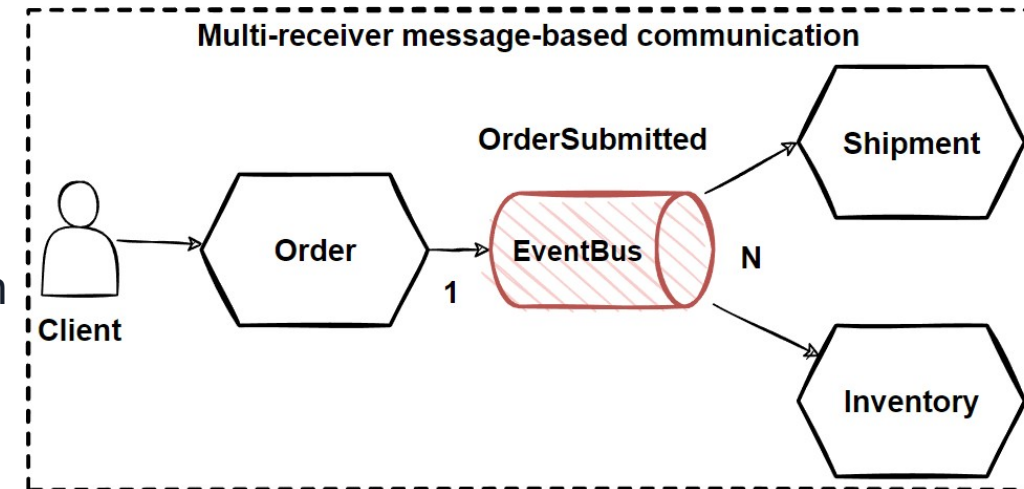
- **one-to-one (queue)** or **point-to-point** communications.
- If you **send 1 request** to the specific consumer, and this operation will take long time, use this **Single-receiver async one-to-one** communication.
- **Single producer** and **single receiver**; implementation as a **Command Pattern**.



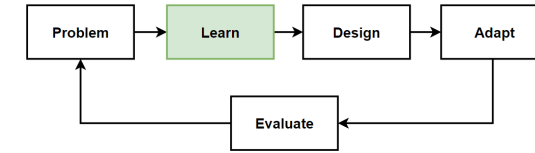
Multiple-receiver Message-based Communication



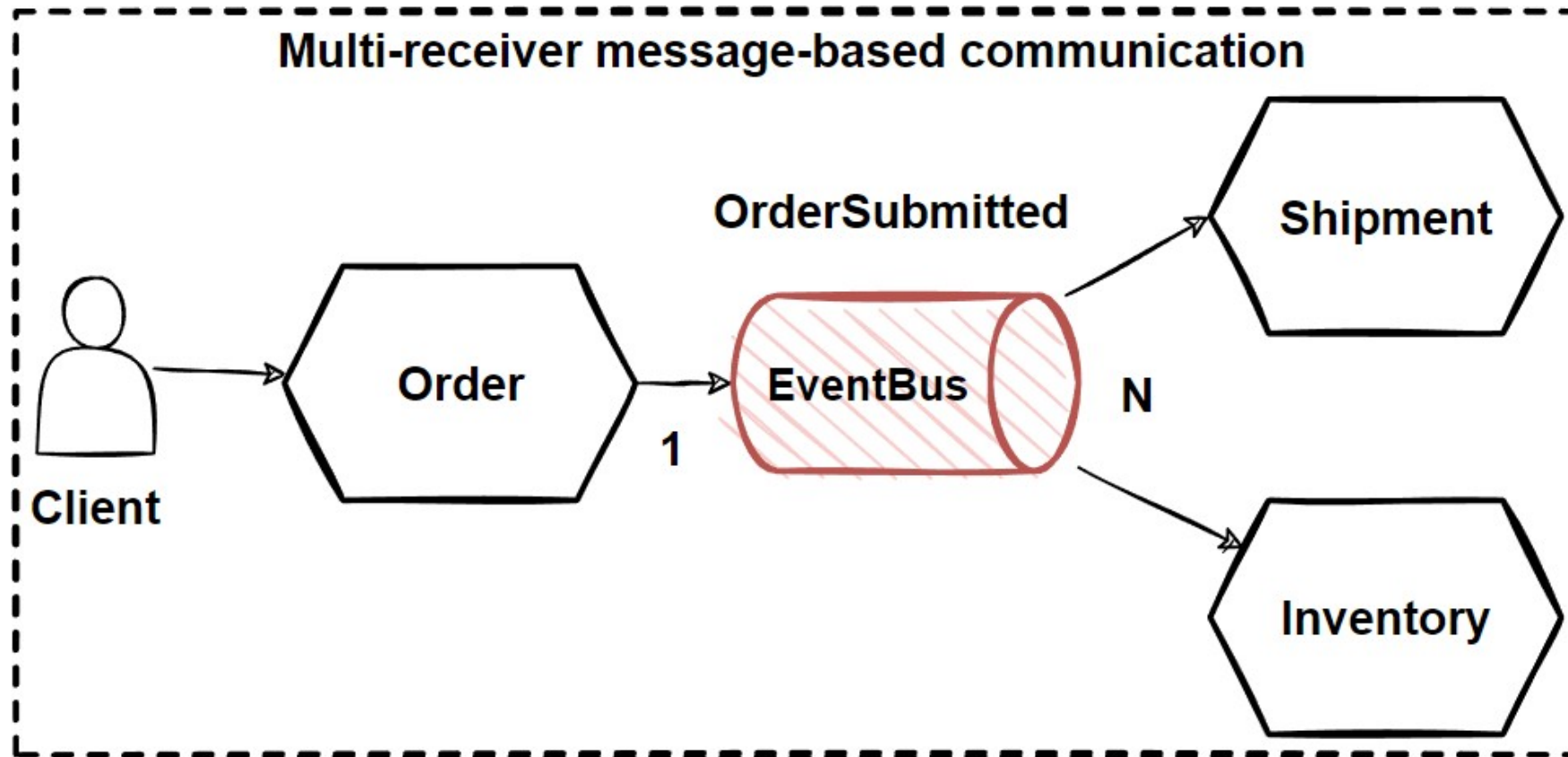
- **Publish/subscribe mechanisms** that has multiple receivers.
- Producer service **publish a message** and it **consumes** from several **microservices** by **subscribing message** on the message broker system.
- **Publisher don't need to know any subscriber**, no any dependency with communication parties.
- **one-to-many (topic)** implementation has Multiple receivers. Each request can be processed **by zero to multiple receivers**.
- Publish/subscribe used in **patterns Event-driven microservices** architecture.
- Message broker system is **publishing events** between multiple microservices, **subscribing these events** in an async way.
- Messages are **available to all subscribers** and the topic can have **more than one subscriber**.
- The message **remains persistent** in a topic until they are deleted.
- **Kafka, RabbitMQ** or **Amazon SNS** and **EventBridge**.



Multiple-receiver Message-based Communication

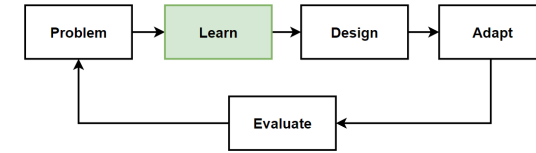


- Microservice **publishes an event** when something happens. Price change in a product microservice: **Price Changed event** can be subscribed from SC microservice in order to update basket price.



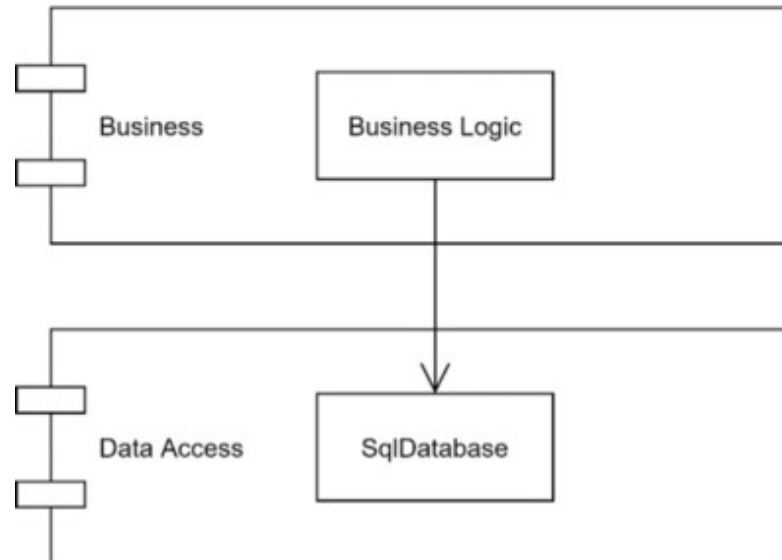
- Event-driven Architecture, **CQRS pattern**, **event storming**, **eventual consistency** principles.

Dependency Inversion Principles (DIP)

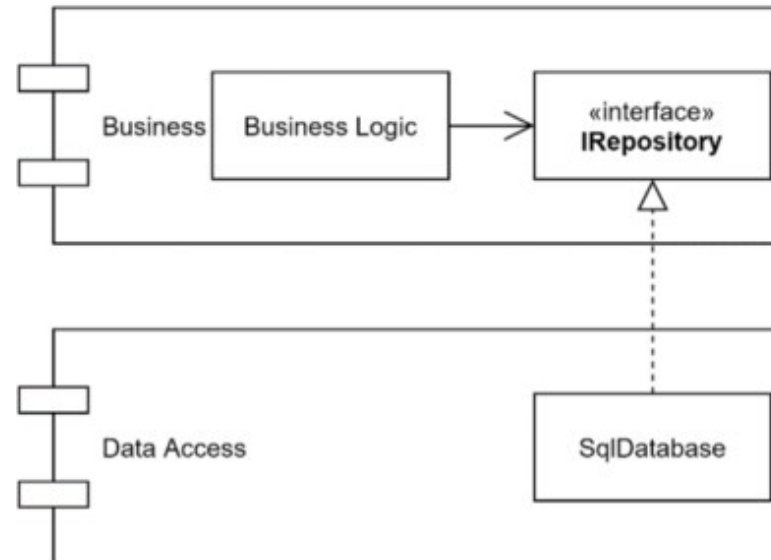


- Provide to broke dependency of classes by inverting dependencies, inject dependent classes via constructor.
- Upper-level modules or classes and lower-level classes must not be dependent on modules.
- Lower-level modules must be dependent on higher-level modules (interfaces of modules).

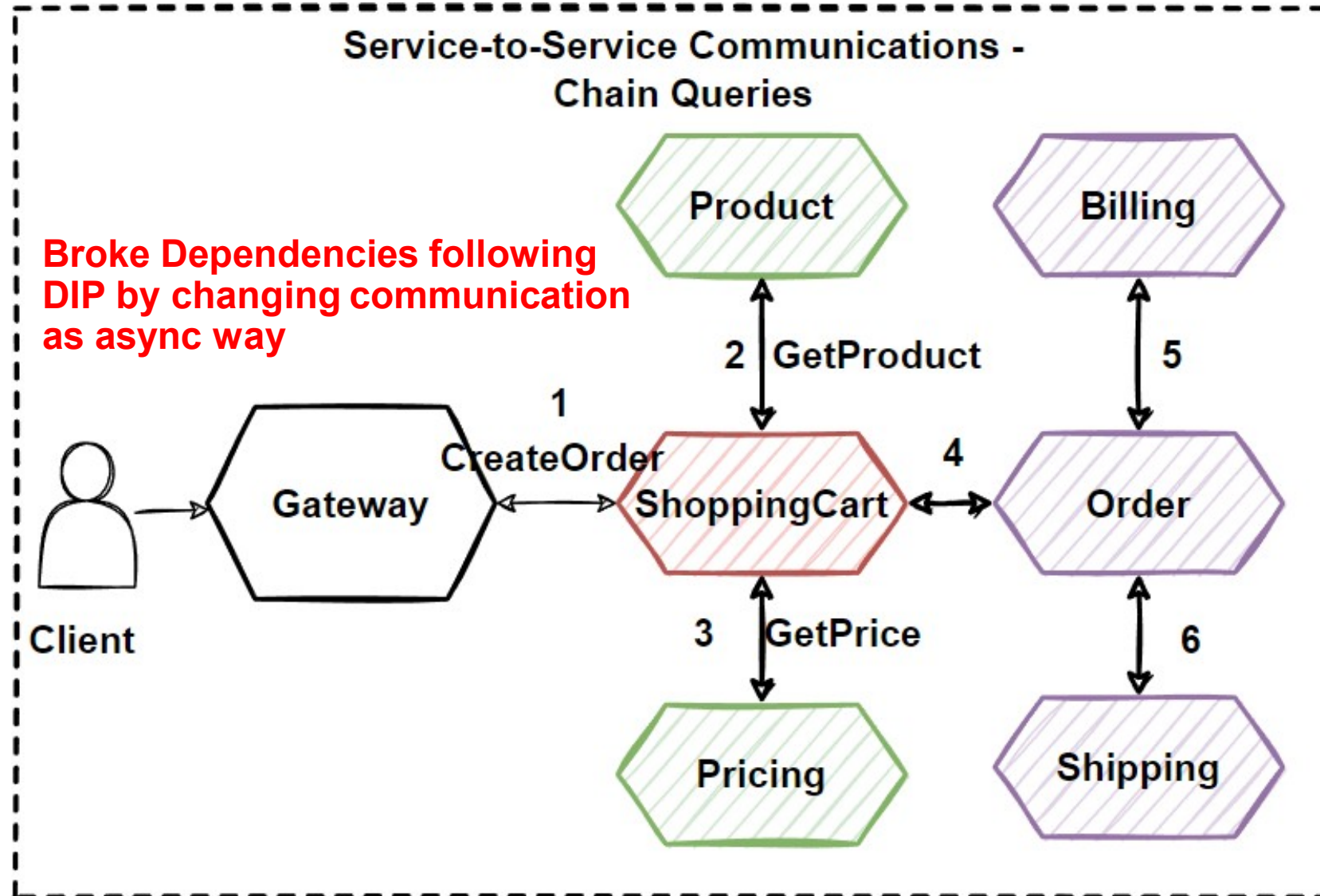
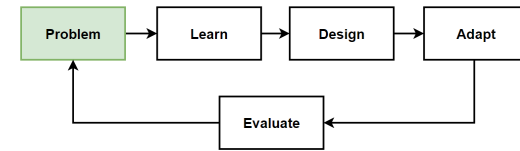
Without Dependency Inversion



With Dependency Inversion

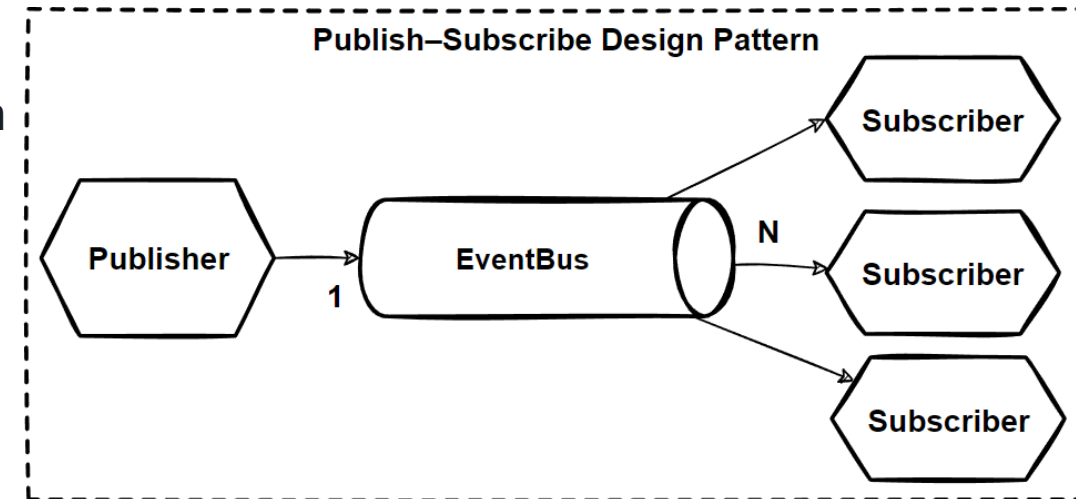
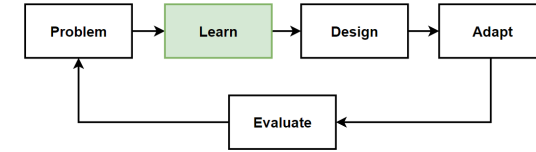


Problem: Chain of request and highly coupled dependent microservices



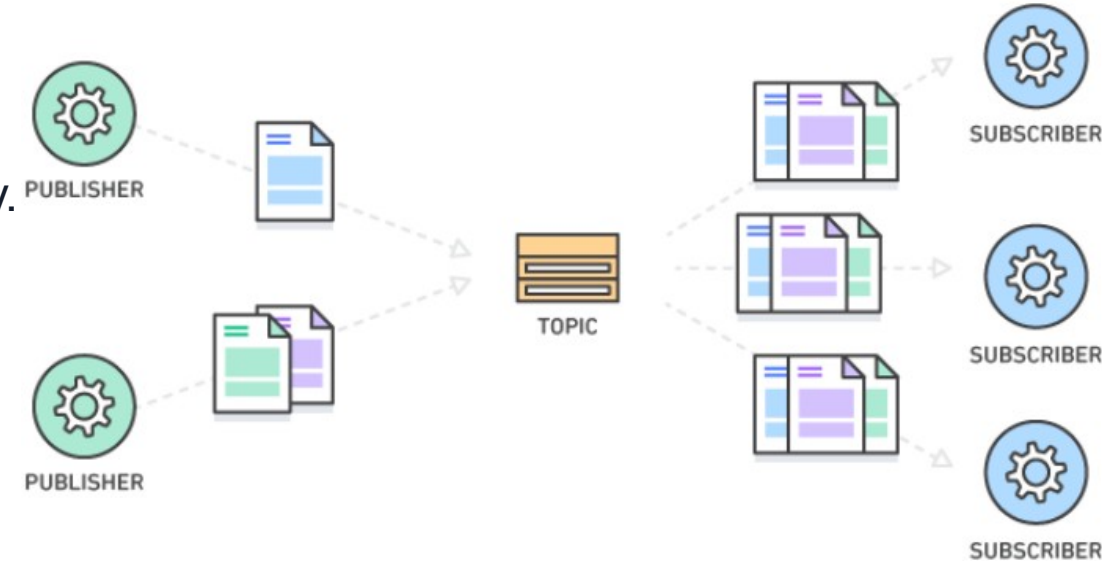
Fan-Out & Message Filtering with Publish/Subscribe Pattern

- **Fan-out** is a messaging pattern; ‘fanned out’ to **multiple destination** in parallel.
- Each of destinations can **work** and **process** this messages in **parallel**.
- **Publisher/subscriber model** to **define a topic** which is **logical access point** to enabling message communication with asynchronously.
- Publisher **sends the message to the topic**, message is immediately **fanned out** to all **subscribers** of this topic.
- Each service can operate and **scale independently** and individually that **completely decoupled** and **asnychronously**.
- The publisher and the subscribers **don't need to know** who is publishing / consuming this message that is broadcasting.
- Deliver the same message to **multiple receivers** is to use the **Fanout Publish/Subscribe Messaging Pattern**.

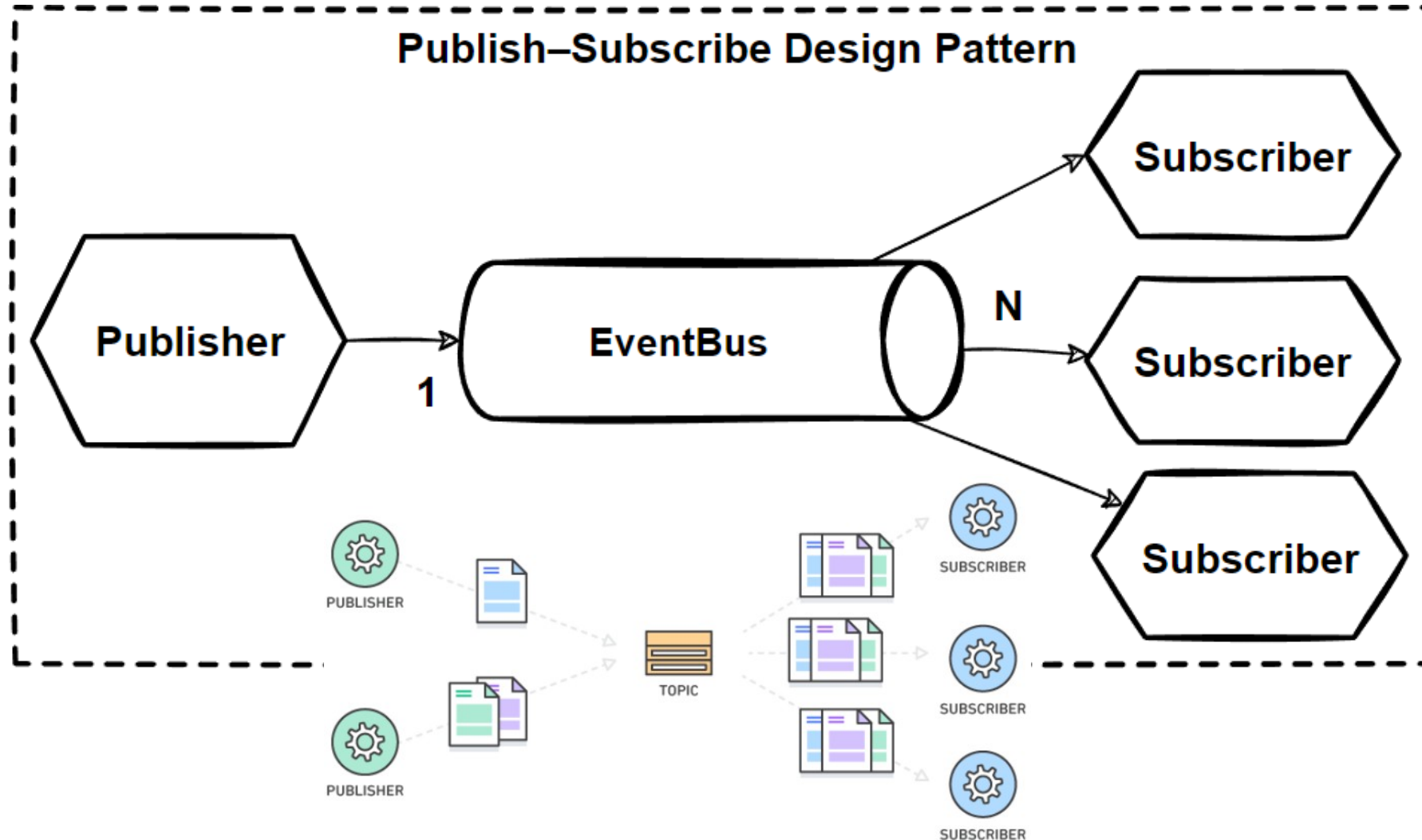
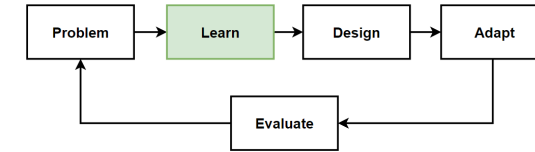


Publish/Subscribe Messaging Pattern

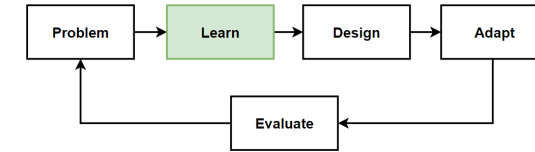
- **Publish/subscribe messaging** is a form of **asynchronous** service-to-service communication.
- Any message **published** to a **topic** is **immediately received** by all of the **subscribers** to the **topic**.
- Enable **event-driven architectures**, and **decouple applications** to increase performance, reliability and scalability.
- Applications are **decoupled** into **smaller, independent building blocks** that are easier to develop, deploy and maintain.
- Publish/Subscribe (Pub/Sub) messaging provides **instant event notifications** for these distributed applications.
- A **message topic** provides a **lightweight mechanism** to broadcast **asynchronous event notifications**.
- All components that subscribe to the topic receive every message, unless a **message filtering policy** is set by the subscriber.



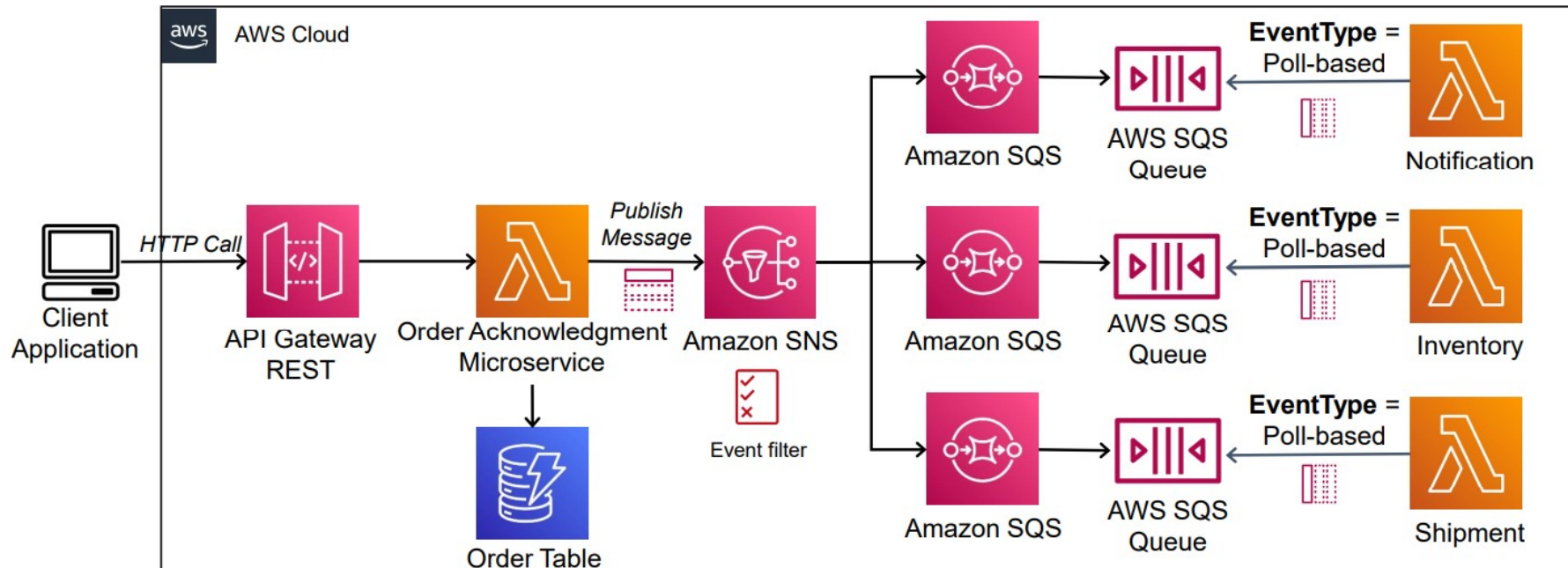
Publish/Subscribe Messaging Pattern-2



Topic-Queue Chaining & Load Balancing Pattern



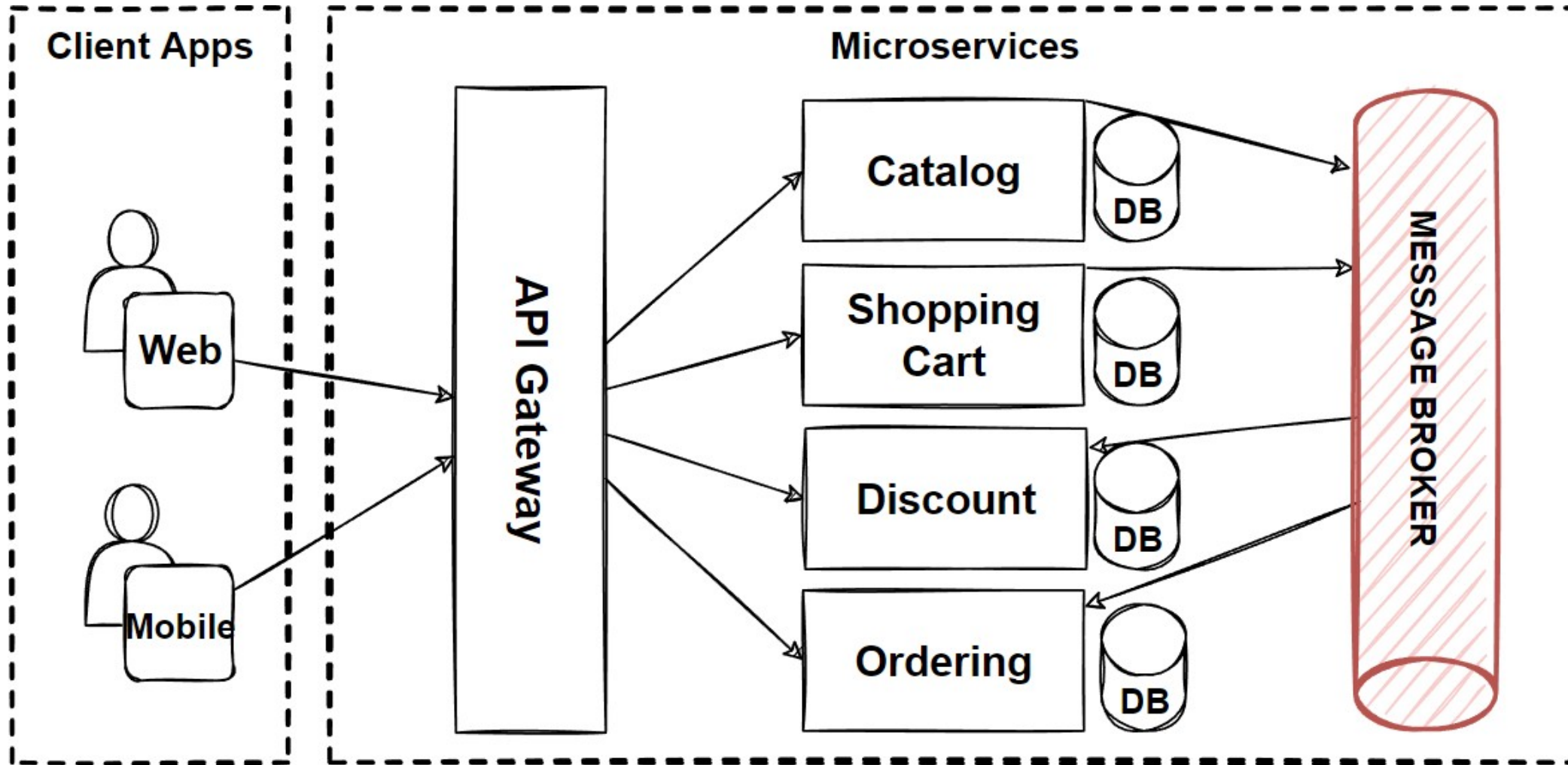
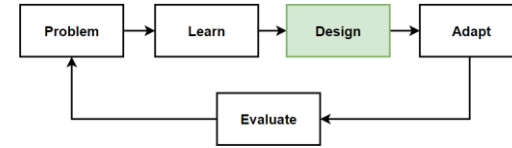
- Use a **queue** that acts as a **buffer** between the **service** to **avoid loss data** if the service to fail.
- **Services can be down** or getting exception or **taken offline** for maintenance, then events will be **loses, disappeared** and can't process after the **subscriber service** is **up and running**.
- Put **Amazon SQS** between EventBridge and Ordering microservices.
- Store this event messages into **SQS queue** with durable and persistent manner, no message will get lost. Queue can act as a **buffering load balancer**.



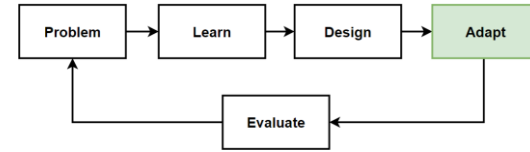
Before Design – What we have in our design toolbox ?

Architectures	Patterns&Principles	Microservices Communications	Microservices Async Communications	FR
• Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns	• HTTP Based RESTful API • GraphQL API • gRPC API • WebSocket API • Gateway Routing Pattern • Gateway Aggregation Pattern • Gateway Offloading Pattern • API Gateway Pattern • Backends for Frontends Pattern-BFF • Service Aggregator Pattern. • Service Registry/Discovery Pattern	• Single-receiver Message-based Communication (one-to-one model) • Multiple-receiver Message-based Communication (one-to-many model-topic) • Dependency Inversion Principles (DIP) • Fan-Out Publish/Subscribe Messaging Pattern • Topic-Queue Chaining & Load Balancing Pattern	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history Non-FR • High Scalability • High Availability • Millions of Concurrent User • Independent

Design: Microservices Architecture with Fan-Out Publish/Subscribe Messaging Pattern



Adapt: Microservice Architecture with Fan-Out Publish/Subscribe Messaging Pattern



Frontend SPAs

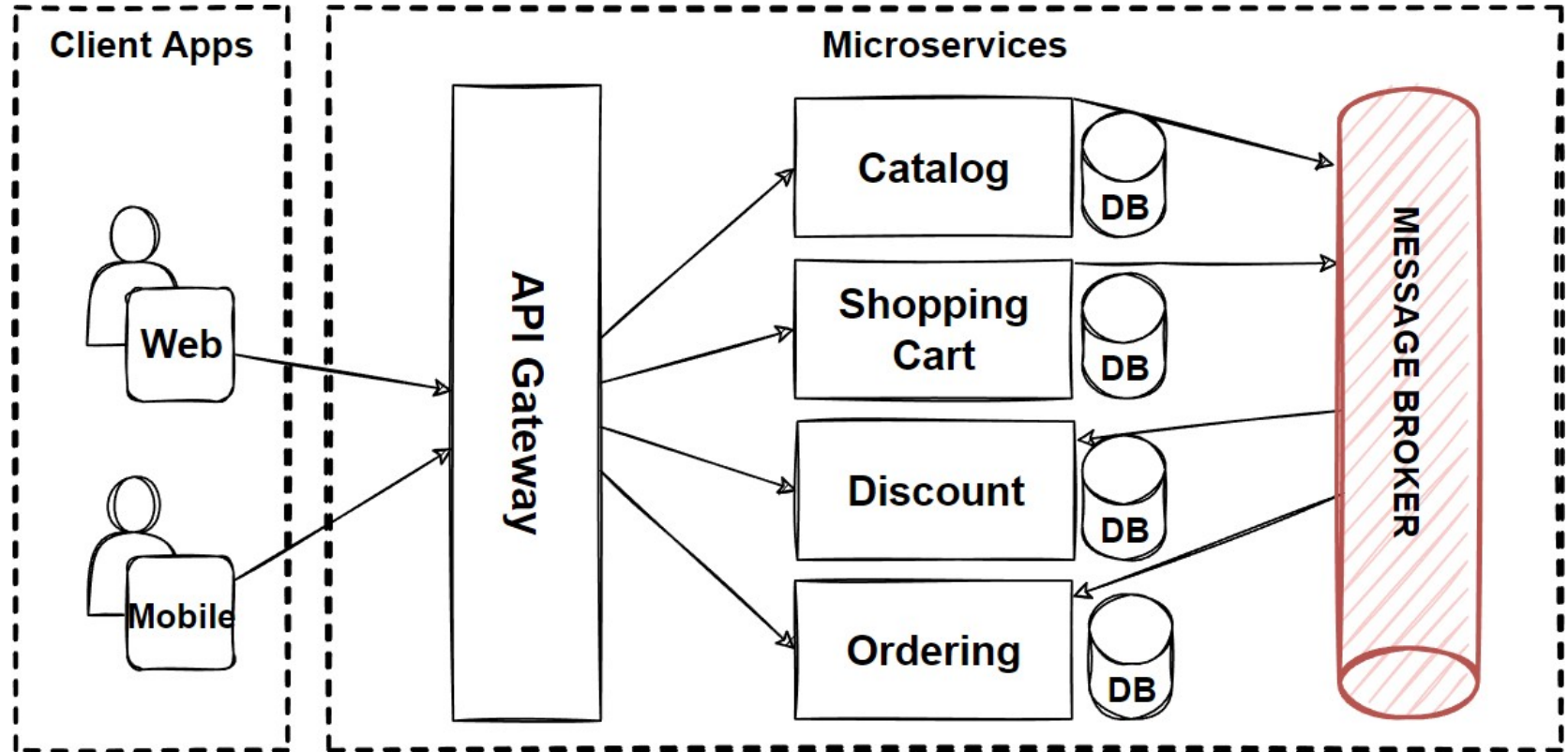
- Angular
- Vue
- React

API Gateways

- Kong Gateway
- Tyk API Gateway
- Express Gateway
- Amazon AWS API Gateway

Message Brokers

- Kafka
- RabbitMQ
- Amazon EventBridge, SNS



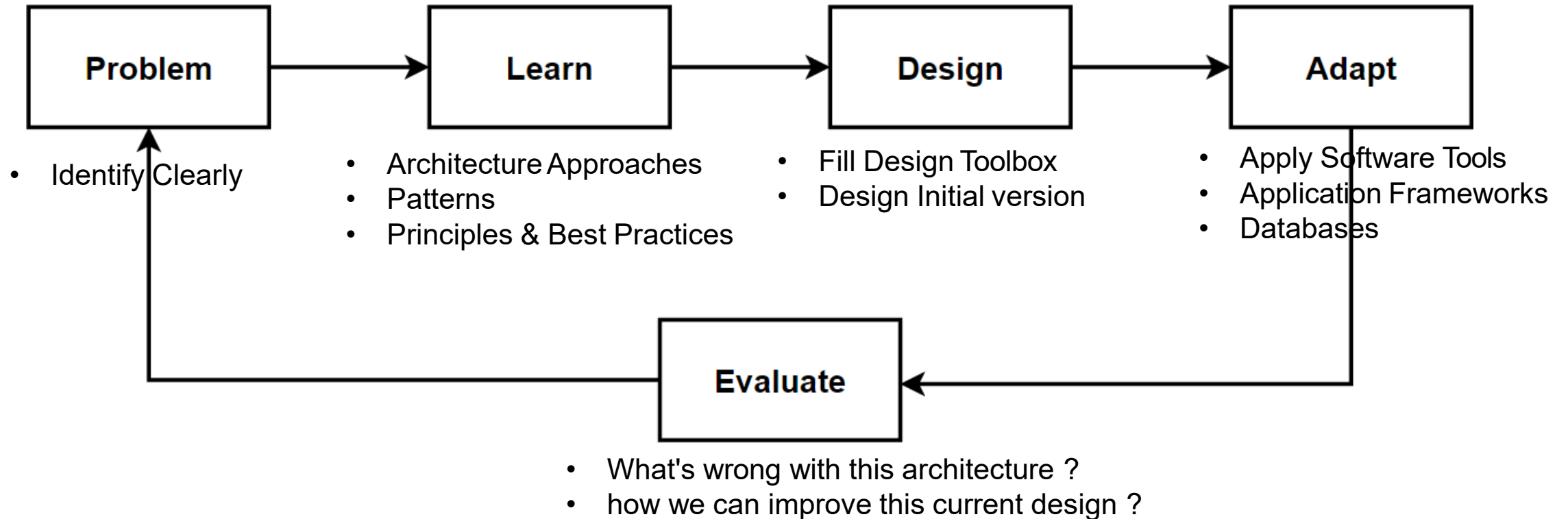
Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS

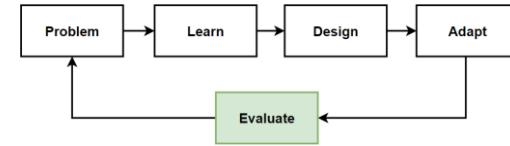
Database

- MongoDB – NoSQL Document DB
- Redis – NoSQL Key-Value Pair
- Postgres – Relational

Way of Learning – The Course Flow



Evaluate: Microservice Architecture with Fan-Out Publish/Subscribe Messaging Pattern

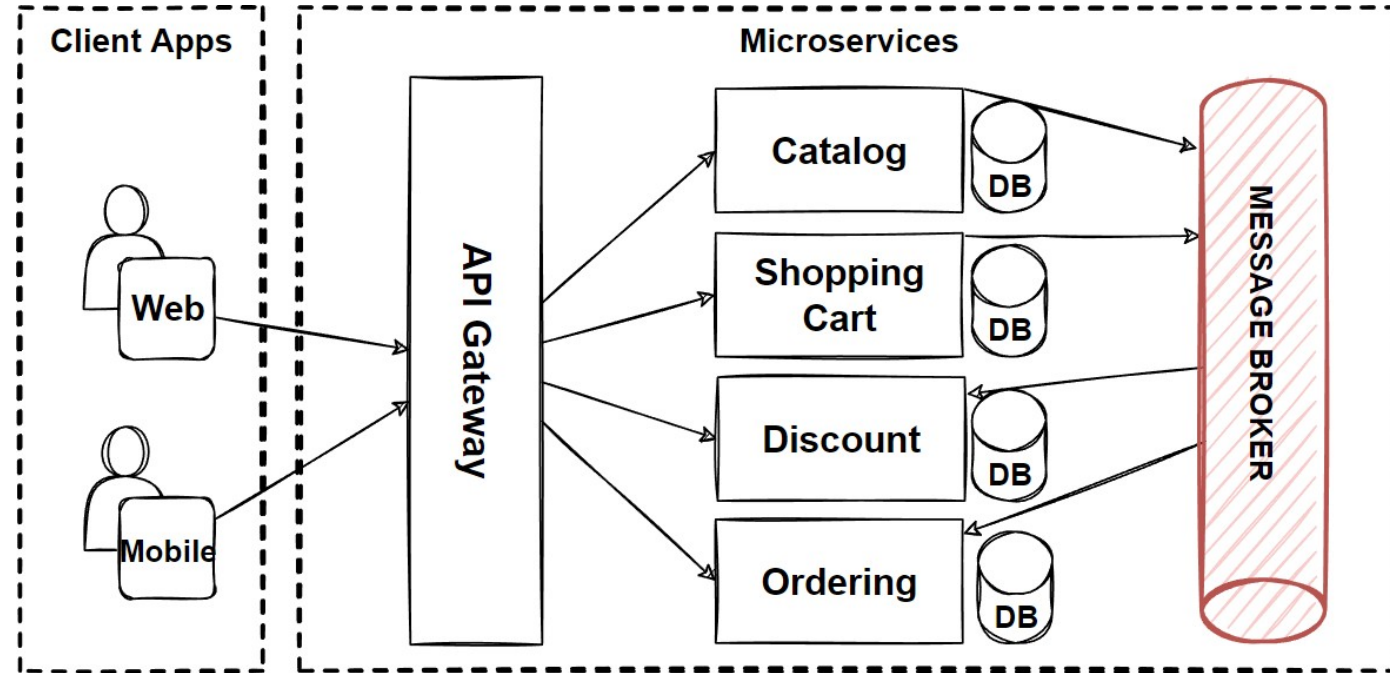


Benefits

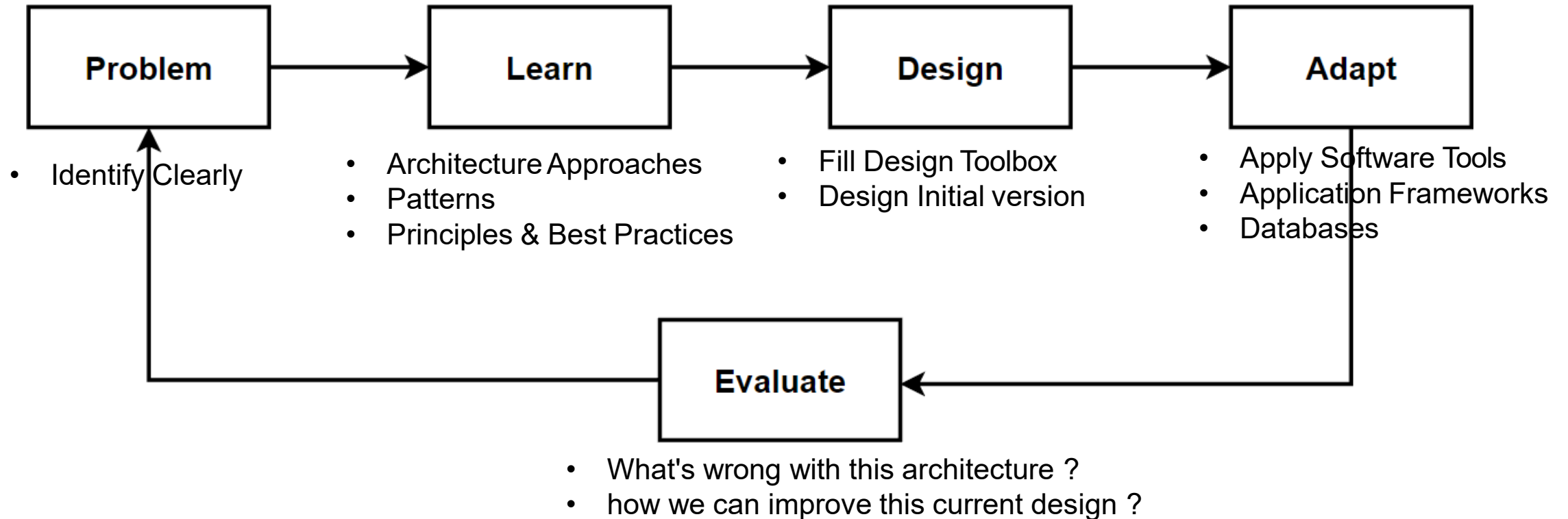
- Adding new subscriber services very easily
- Increase flexible scalability
- Distribute received messages at scale
- Event-driven Microservices

Drawbacks

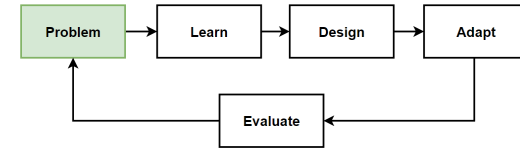
- Single Point of Failure - Message Broker
- Hard to Debugging
- Not Good for FIFO and exact-once requirements can cause performance and scalability problems



Way of Learning – The Course Flow



Problem: Database Bottlenecks when Scaling



Considerations

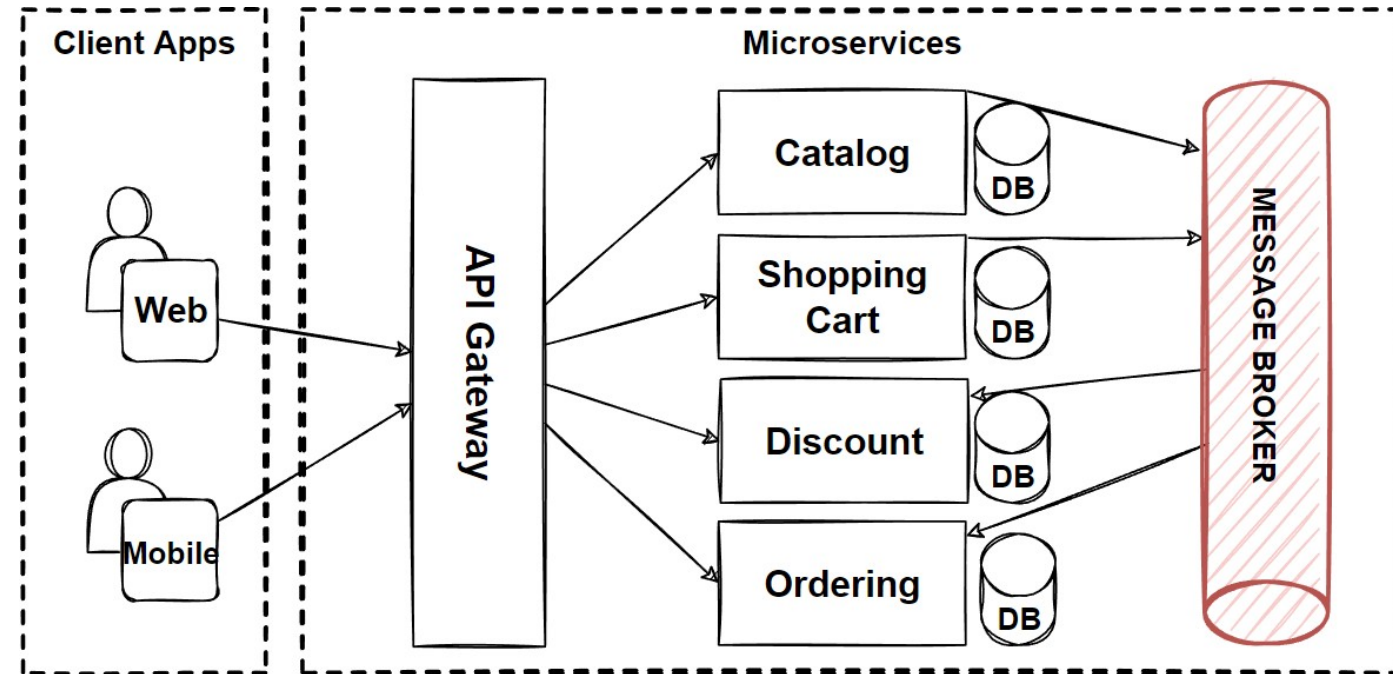
- State-less or state-ful services
- **Are they can scale independently ?**

Problems

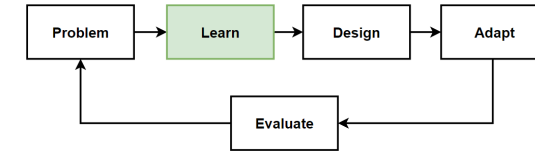
- Database are stateful service
- Scaling stateful services are not easy
- Vertical scaling has limits need to scale horizontally

Solutions

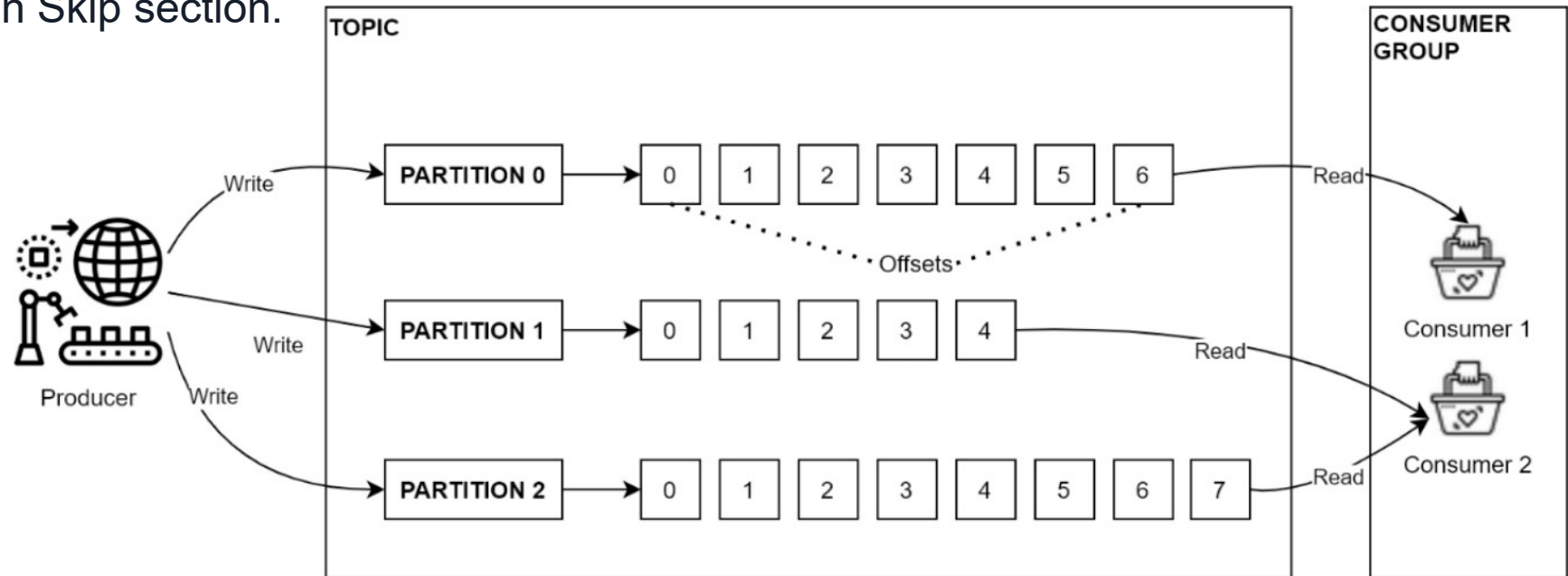
- Scale Stateful Application Horizontal Scaling
- Service and Data Partitioning along Business Boundaries - Shards/Pods
- Use NoSQL Database to gain easy partitioning features



Optional Section: Learning Kafka and RabbitMQ Architecture



- Message broker system that provide to communicate Asynchronously between microservices.
- Kafka and RabbitMQ is good example of distributed architecture that is good to learn with microservices.
- Optional Section, you can Skip section.



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**